

Hoja de Trabajo #9 – Compresor y Descompresor de Huffman

Integrantes: Diego Quan & Diego Gonzáles

Fecha: 29/04/2025

1. Objetivo

Implementar en Java el algoritmo de codificación de Huffman para comprimir y descomprimir archivos de texto plano, usando un árbol binario óptimo que minimice la longitud total de la representación binaria. El enfoque principal es generar códigos de longitud variable según la frecuencia de los caracteres, de modo que los que aparecen con mayor frecuencia reciban códigos más cortos, garantizando así una compresión eficiente.

2. Diseño e implementación

- Conteo de frecuencias:
 - Se lee el archivo completo en un String.
 - Se usa `HashMap<Character,Integer>` para acumular la cantidad de ocurrencias de cada carácter.
- Construcción del árbol de Huffman:
 - Cada carácter con su frecuencia se envuelve en un `HuffmanNode`.
 - `PriorityQueue<HuffmanNode>` ordena nodos por frecuencia.
 - Se combinan los dos nodos de menor frecuencia recursivamente hasta obtener la raíz.
- Generación de códigos binarios:
 - Recorrido recursivo del árbol: "0" para rama izquierda, "1" para rama derecha.
 - Al llegar a una hoja, el prefijo acumulado se asigna como código del carácter.
- Serialización y almacenamiento:
 - `.hufftree`: recorrido en preorden con boolean (false=nodo interno; true+char=hoja).
 - `.huff`: flujo puro de bits agrupados en bytes, sin metadata extra.

2.1 Uso en consola y sistema de archivos

Al ejecutar la opción de compresión, la aplicación indica en consola un mensaje de éxito sin mostrar detalles internos de bits, pero crea los archivos `".huff"` y `".hufftree"` en la carpeta de trabajo especificada.

Estos archivos pueden visualizarse y abrirse directamente en el explorador de archivos o con herramientas de edición hexadecimal para verificar su contenido.

De manera similar, al seleccionar descompresión, el programa genera el archivo restaurado (ejemplo1_decoded.txt) que coincide exactamente con el original, visible tras refrescar el directorio.

Repositorio

<https://github.com/dquan123/HDT-9.git>

3. Pruebas realizadas

Archivo	Original	.huff	.hufftree	Ratio
ABRACADABRA	11 B	3 B	19 B	27%
aaaaaaaaaaa	10 B	2 B	7 B	20%
hello world	11 B	5 B	27 B	45%
To be or not to be	18 B	7 B	31 B	39%

4. Conclusión

La solución implementada cumple con los objetivos y requisitos: el uso de PriorityQueue garantiza la construcción óptima del árbol de Huffman; la serialización en preorden facilita la reconstrucción sin ambigüedad; y la clase BitUtils permite manejar flujos de bits de manera transparente para el usuario. Las pruebas realizadas demuestran que Huffman ofrece compresiones significativas en textos con alta repetición, manteniendo integridad total al descomprimir. El repositorio en GitHub contiene todo el código fuente, los archivos de prueba y las instrucciones de uso.